

Sorting Algorithms in C++ - Beginner Notes

These notes cover five classic sorting algorithms:

1. [Bubble Sort](#)
 2. [Selection Sort](#)
 3. [Insertion Sort](#)
 4. [Merge Sort](#)
 5. [Quick Sort](#)
-

0. Ideas You Need First

0.1 What is sorting?

Sorting means rearranging items into an order. For `{5, 1, 4, 2, 8}` the ascending order is `{1, 2, 4, 5, 8}`.

0.2 What is an array and an index?

```
int arr[5] = {5, 1, 4, 2, 8};
```

Index	0	1	2	3	4
Value	5	1	4	2	8

- `arr[0]` is 5, `arr[4]` is 8.
- The first index is always 0. The last index is `n - 1` (where `n` is the number of elements).

0.3 How to swap two values (without `std::swap`)

To swap `a` and `b` you need a third box to hold one value temporarily. Without it, one value gets overwritten and lost.

```
int temp = a;    // 1. save a
a = b;           // 2. copy b into a
b = temp;        // 3. put the saved value into b
```

Think of two cups, one with tea and one with juice. To swap them you need an empty third cup.

0.4 Big-O in one minute

Big-O describes how the number of steps grows when the array gets bigger (`n` = number of elements).

Notation	Meaning for beginners	Example with <code>n = 1000</code>
$O(n)$	Grows in a straight line	about 1,000 steps
$O(n \log n)$	Grows a bit faster than a line (fast for sorting)	about 10,000 steps
$O(n^2)$	Grows very quickly (slow for big data)	about 1,000,000 steps

0.5 Two words used in every section

- **Pass**: one full trip of the outer loop.
- **Sorted part / unsorted part**: while sorting, the array is split into a section that is already in the right place and a section that still needs work.

0.6 Two properties of sorting algorithms

- **Stable**: if two elements are equal, they keep their original relative order. (Matters when sorting things like students by score.)
 - **In-place**: it sorts using (almost) no extra array. Merge Sort is not in-place because it needs a temporary array.
-

1. Bubble Sort

1.1 The idea

Look at neighbours, two at a time. If the left one is bigger than the right one, swap them. Repeat from left to right.

After one full trip, the **biggest value has "bubbled up"** to the end of the array (like a bubble rising to the surface). Then repeat, but each time you can ignore the end because it is already correct.

1.2 Steps

1. Compare `arr[0]` and `arr[1]`. If `arr[0] > arr[1]`, swap them.
2. Move one step right and compare `arr[1]` and `arr[2]`, and so on until the end of the unsorted part.
3. The largest value is now at the end. Shrink the unsorted part by 1.
4. Repeat steps 1 to 3 until a whole pass makes **no swaps** (it means everything is already sorted).

1.3 Source code

```
#include <iostream>
using namespace std;

int main() {
    const int n = 5;
    int arr[n] = {5, 1, 4, 2, 8};

    cout << "Before sorting: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    // Outer loop: each pass pushes the biggest remaining value to the end
    for (int pass = 0; pass < n - 1; pass++) {

        bool swapped = false;    // did we swap anything during this pass?

        // Inner loop: compare neighbours
        // "n - 1 - pass" because the last "pass" elements are already sorted
        for (int i = 0; i < n - 1 - pass; i++) {

            if (arr[i] > arr[i + 1]) {        // left is bigger than right?
                int temp = arr[i];           // swap them
                arr[i] = arr[i + 1];
                arr[i + 1] = temp;
                swapped = true;
            }
        }
    }
}
```

```

    }

    // No swaps in a whole pass means the array is already sorted
    if (swapped == false) {
        break;
    }
}

cout << "After sorting: ";
for (int i = 0; i < n; i++) {
    cout << arr[i] << " ";
}
cout << endl;

return 0;
}

```

1.4 How the code works, part by part

`for (int pass = 0; pass < n - 1; pass++)` This is the outer loop. It counts the passes. With `n` elements you need at most `n - 1` passes, because each pass fixes one value in its final place and the last remaining value is automatically correct.

`bool swapped = false;` A flag (a yes/no note). At the beginning of every pass we assume "nothing was swapped yet".

`for (int i = 0; i < n - 1 - pass; i++)` The inner loop walks through neighbouring pairs (`arr[i]`, `arr[i + 1]`).

- We stop at `n - 1 - pass` because we look at `arr[i + 1]`, and we must not go past the last index.
- The `- pass` part is the optimization: after pass 0 the last element is fixed, after pass 1 the last two are fixed, and so on. No need to check them again.

`if (arr[i] > arr[i + 1])` If the pair is in the wrong order, swap using the `temp` variable and set `swapped = true`.

`if (swapped == false) break;` If we walked through the whole unsorted part and never swapped, then every neighbour pair is in order, so the array is sorted. `break` leaves the loop early. This makes Bubble Sort very fast on already sorted input.

1.5 Dry run (trace) with {5, 1, 4, 2, 8}

Pass 0 (compare pairs at index 0-1, 1-2, 2-3, 3-4)

Step	Compare	Swap?	Array after step
1	5 and 1	yes	1 5 4 2 8
2	5 and 4	yes	1 4 5 2 8
3	5 and 2	yes	1 4 2 5 8
4	5 and 8	no	1 4 2 5 8

The biggest value (8) is at the end. Result of pass 0: 1 4 2 5 8

Pass 1 (now only compare pairs at index 0-1, 1-2, 2-3)

Step	Compare	Swap?	Array after step
1	1 and 4	no	1 4 2 5 8

Step	Compare	Swap?	Array after step
2	4 and 2	yes	1 2 4 5 8
3	4 and 5	no	1 2 4 5 8

Result of pass 1: 1 2 4 5 8

Pass 2 (compare pairs at index 0-1, 1-2)

Nothing needs swapping, so `swapped` stays `false` and we `break`. Final: 1 2 4 5 8

1.6 Summary

Property	Value
Best case	$O(n)$ (already sorted, thanks to <code>swapped</code>)
Average / Worst	$O(n^2)$
Extra memory	$O(1)$
Stable	Yes
Good for	Learning, tiny or nearly sorted data

2. Selection Sort

2.1 The idea

Split the array into a **sorted part** (left, starts empty) and an **unsorted part** (right, starts as everything).

Each round: **search** the unsorted part for the smallest value, then **place** it at the front of the unsorted part (by swapping). The sorted part grows by one each round.

Real-life example: picking cards from a table. Each time, find the smallest card in the pile on the table and put it at the end of the row in your hand.

2.2 Steps

1. Assume the first unsorted element is the smallest. Remember its index.
2. Scan the rest of the unsorted part. Whenever you see a smaller value, update the remembered index.
3. After scanning, swap the smallest found with the first unsorted element.
4. The sorted part is now one element longer. Repeat until only one element is left.

2.3 Source code

```
#include <iostream>
using namespace std;

int main() {
    const int n = 5;
    int arr[n] = {64, 25, 12, 22, 11};

    cout << "Before sorting: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
}
```

```

cout << endl;

// i = the position that we are trying to fill with the correct value
// (everything to the left of i is already sorted)
for (int i = 0; i < n - 1; i++) {

    int minIndex = i;    // assume the smallest is at position i

    // Search the unsorted part (from i + 1 to the end) for anything smaller
    for (int j = i + 1; j < n; j++) {
        if (arr[j] < arr[minIndex]) {
            minIndex = j;        // found a new smallest, remember WHERE it is
        }
    }

    // Put the smallest value at position i by swapping
    if (minIndex != i) {        // no need to swap a value with itself
        int temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }
}

cout << "After sorting: ";
for (int i = 0; i < n; i++) {
    cout << arr[i] << " ";
}
cout << endl;

return 0;
}

```

2.4 How the code works, part by part

`for (int i = 0; i < n - 1; i++)` `i` is the "slot" we are filling. In round `i = 0` we find the smallest of the whole array and put it in slot 0. In round `i = 1` we find the smallest of the rest and put it in slot 1, and so on. We stop at `n - 1` because when only one element remains, it is automatically in the correct place.

`int minIndex = i;` We keep the **index** of the smallest value (not the value itself). Why? Because we need to know *where* it is so we can swap it later.

`for (int j = i + 1; j < n; j++)` Scans everything to the right of slot `i`. If `arr[j]` is smaller than the current smallest `arr[minIndex]`, we update `minIndex = j`.

The swap After the scan, `minIndex` points to the smallest value in the unsorted part. Swap it with `arr[i]`. Now slot `i` holds its final value and will never change again.

2.5 Dry run (trace) with {64, 25, 12, 22, 11}

In the tables below, the part before the `|` is the sorted part.

Round (i)	Unsorted part scanned	Smallest found	Swap	Array after round
start	-	-	-	\ 64 25 12 22 11
0	64 25 12 22 11	11 (index 4)	swap arr[0] and arr[4]	11 \ 25 12 22 64
1	25 12 22 64	12 (index 2)	swap arr[1] and arr[2]	11 12 \ 25 22 64
2	25 22 64	22 (index 3)	swap arr[2] and arr[3]	11 12 22 \ 25 64
3	25 64	25 (index 3)	none (already in place)	11 12 22 25 \ 64

2.6 Summary

Property	Value
Best / Avg / Worst	$O(n^2)$ always (it always scans, even if sorted)
Extra memory	$O(1)$
Stable	No (the long-distance swap can change the order of equal values)
Good for	Learning, and when swaps are expensive (it makes at most $n - 1$ swaps)

3. Insertion Sort

3.1 The idea

This is how most people sort playing cards in their hand.

The left part of the array is always sorted. Take the next card (the **key**) from the unsorted part, and **slide it left** into the correct place inside the sorted part. Cards bigger than the key shift one step right to make room.

3.2 Steps

1. Treat `arr[0]` as a sorted part of size 1.
2. Take the next element as the `key`.
3. Compare `key` with elements of the sorted part, from right to left. While the element is bigger than `key`, shift it one position to the right.
4. When you find an element that is not bigger (or you fall off the left edge), drop `key` into the gap.
5. Repeat for every remaining element.

3.3 Source code

```
#include <iostream>
using namespace std;

int main() {
    const int n = 5;
    int arr[n] = {12, 11, 13, 5, 6};

    cout << "Before sorting: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    // Start at index 1 because index 0 alone is already "sorted"
    for (int i = 1; i < n; i++) {

        int key = arr[i];    // the card we are about to insert
        int j = i - 1;      // j starts at the last element of the sorted part

        // Move left while we are still inside the array
        // AND the element is bigger than key
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];    // shift the bigger element one step right
            j--;                    // look at the next element on the left
        }

        arr[j + 1] = key;
    }
}
```

```

        // j stopped at an element that is <= key (or at -1),
        // so the correct gap is j + 1
        arr[j + 1] = key;
    }

    cout << "After sorting: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    return 0;
}

```

3.4 How the code works, part by part

`int key = arr[i];` We copy the value out first. This is important: the shifting below will overwrite `arr[i]`, so `key` is our safe copy.

`int j = i - 1;` `j` points to the rightmost element of the sorted part. We will walk left from here.

`while (j >= 0 && arr[j] > key)` Two conditions, both must be true to keep shifting:

- `j >= 0`: we have not fallen off the left edge of the array. (This check must come first, otherwise `arr[-1]` would be read, which is invalid.)
- `arr[j] > key`: the element is bigger than `key`, so `key` must go to its left.

`arr[j + 1] = arr[j];` Shift the bigger element one step to the right. This overwrites the old spot of `key` (first time) or the previous duplicate (later times). No data is lost because `key` is saved.

`arr[j + 1] = key;` The loop ended, so `arr[j]` is not bigger than `key` (or `j` is `-1`). The gap is at `j + 1`. Drop `key` in.

3.5 Dry run (trace) with {12, 11, 13, 5, 6}

i = 1, key = 11 (sorted part: 12)

- `12 > 11`? yes, shift 12 right: 12 12 13 5 6, `j` becomes `-1`.
- `j` is `-1`, loop stops. Place key at index 0: 11 12 13 5 6

i = 2, key = 13 (sorted part: 11 12)

- `12 > 13`? no. Loop stops immediately. Place key back at index 2 (no change): 11 12 13 5 6

i = 3, key = 5 (sorted part: 11 12 13)

- `13 > 5`? yes, shift: 11 12 13 13 6
- `12 > 5`? yes, shift: 11 12 12 13 6
- `11 > 5`? yes, shift: 11 11 12 13 6
- `j = -1`, stop. Place key at index 0: 5 11 12 13 6

i = 4, key = 6 (sorted part: 5 11 12 13)

- `13 > 6`? yes, shift: 5 11 12 13 13
- `12 > 6`? yes, shift: 5 11 12 12 13
- `11 > 6`? yes, shift: 5 11 11 12 13
- `5 > 6`? no, stop. `j = 0`, so place key at index 1: 5 6 11 12 13

Final: 5 6 11 12 13

3.6 Summary

Property	Value
Best case	$O(n)$ (already sorted: the <code>while</code> never runs)
Average / Worst	$O(n^2)$
Extra memory	$O(1)$
Stable	Yes
Good for	Small arrays, nearly sorted data, live data arriving one by one

4. Merge Sort

4.1 The idea: Divide and Conquer

Merge Sort is built on one simple fact: **merging two already-sorted lists is easy and fast.**

Example: merge `[3, 27]` and `[9, 38]`. Look at the front of each list, take the smaller one, repeat.

```
[3, 27]  [9, 38]  -> take 3
[27]     [9, 38]  -> take 9
[27]     [38]     -> take 27
[]       [38]     -> take 38
Result: [3, 9, 27, 38]
```

So the plan is:

1. **Divide:** split the array into halves again and again until each piece has 1 element (a single element is always sorted).
2. **Merge:** combine pieces back together, two sorted pieces at a time, until one big sorted array is left.

4.2 Recursive version vs the version used here

The textbook version splits from the top down using a function that calls itself:

```
sort(whole array)
-> sort(left half)
-> sort(right half)
-> merge(left half, right half)
```

We are not allowed to have extra functions, so we do the same work **from the bottom up** with loops. Start with pieces of size 1 and merge neighbours into pieces of size 2. Then merge neighbours into size 4, then 8, and so on. The size of the piece is called `width`.

Picture for `{38, 27, 43, 3, 9, 82, 10}`:

```
width = 1:  [38] [27] [43] [3] [9] [82] [10]
              merge pairs
width = 2:  [27 38] [3 43] [9 82] [10]
              merge pairs
width = 4:  [3 27 38 43] [9 10 82]
              merge pairs
width = 8:  [3 9 10 27 38 43 82]      (done, width >= n)
```

Notice the pieces do not need to be a perfect power of 2. The last piece is just shorter, and the code handles that.

4.3 Source code

```
#include <iostream>
using namespace std;

int main() {
    const int n = 7;
    int arr[n] = {38, 27, 43, 3, 9, 82, 10};
    int temp[n];    // helper array used while merging

    cout << "Before sorting: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    // width = size of each already-sorted piece: 1, 2, 4, 8, ...
    for (int width = 1; width < n; width = width * 2) {

        // Go through the array, taking two neighbouring pieces at a time
        for (int left = 0; left < n; left = left + 2 * width) {

            // Piece 1 = [left, mid)    Piece 2 = [mid, right)
            int mid = left + width;
            int right = left + 2 * width;

            // The last pieces may run past the end of the array, so clamp them
            if (mid > n) {
                mid = n;
            }
            if (right > n) {
                right = n;
            }

            int i = left;    // reader position in piece 1
            int j = mid;     // reader position in piece 2
            int k = left;    // writer position in temp

            // MERGE: repeatedly copy the smaller front value into temp
            while (i < mid && j < right) {
                if (arr[i] <= arr[j]) {
                    temp[k] = arr[i];
                    i++;
                } else {
                    temp[k] = arr[j];
                    j++;
                }
                k++;
            }

            // One piece ran out. Copy whatever is left of the other piece.
            while (i < mid) {
                temp[k] = arr[i];
                i++;
                k++;
            }
            while (j < right) {
                temp[k] = arr[j];
                j++;
                k++;
            }

            // Copy the merged result from temp back into arr
```

```

        for (int x = left; x < right; x++) {
            arr[x] = temp[x];
        }
    }
}

cout << "After sorting: ";
for (int i = 0; i < n; i++) {
    cout << arr[i] << " ";
}
cout << endl;

return 0;
}

```

4.4 How the code works, part by part

`int temp[n];` Merging needs a scratch area. We write the merged result into `temp` (so we do not overwrite values we still need to read), then copy it back.

Outer loop: `width = 1, 2, 4, ...` `width` is the length of each sorted piece right now. At the start it is 1 (every single element is a sorted piece). Each round doubles it. The loop ends once `width >= n`, because then one piece covers the whole array.

Middle loop: `left` jumps in steps of `2 * width` because each merge handles **two** pieces. `left` is where the current pair begins.

`mid` and `right`

- Piece 1 covers indexes `left` up to (but not including) `mid`.
- Piece 2 covers indexes `mid` up to (but not including) `right`.
- if (`mid > n`) `mid = n`; and if (`right > n`) `right = n`; handle the leftover pieces at the end. If `mid == n`, piece 2 is empty and the merge simply copies piece 1 unchanged.

The merge while (`i < mid && j < right`) `i` reads piece 1, `j` reads piece 2, `k` writes into `temp`. Each time, we compare the two front values and copy the smaller one, then move that reader and the writer forward. Using `<=` (not `<`) means that for equal values the left one goes first, which keeps Merge Sort **stable**.

Two "leftover" loops When the first `while` ends, one piece is used up. The other piece still has values, and they are already sorted and all bigger than what was copied, so we copy them over as they are. Only one of the two loops will actually run.

`for (int x = left; x < right; x++) arr[x] = temp[x];` Copy the merged section back so the next round works on the updated `arr`.

4.5 Dry run (trace) with {38, 27, 43, 3, 9, 82, 10}

`width = 1` (merge single elements into sorted pairs)

left	Piece 1	Piece 2	Merged result
0	38	27	27 38
2	43	3	3 43
4	9	82	9 82
6	10	(empty, since mid clamped to 7)	10

Array now: 27 38 3 43 9 82 10

width = 2 (merge pairs into groups of four)

left	Piece 1	Piece 2	Merged result
0	27 38	3 43	3 27 38 43
4	9 82	10	9 10 82

Array now: 3 27 38 43 9 10 82

Detail of the first merge [27 38] with [3 43] :

Compare	Copy	temp so far
27 vs 3	3	3
27 vs 43	27	3 27
38 vs 43	38	3 27 38
piece 1 is empty, copy leftover 43	43	3 27 38 43

width = 4 (merge the last two groups)

left	Piece 1	Piece 2	Merged result
0	3 27 38 43	9 10 82	3 9 10 27 38 43 82

Array now: 3 9 10 27 38 43 82

width = 8: 8 < 7 is false, the loop ends. Sorted.

4.6 Summary

Property	Value
Best / Avg / Worst	O(n log n) always
Extra memory	O(n) (the temp array)
Stable	Yes
Good for	Large data, when you need a guaranteed speed and stability

5. Quick Sort

5.1 The idea: pick a pivot and partition

Quick Sort is also Divide and Conquer, but the work is done while **splitting** rather than while merging.

1. Pick one element as the **pivot**. (Here we always pick the last element of the current range.)
2. **Partition:** rearrange the range so that
 - everything **smaller** than the pivot is on its left,
 - everything **bigger** is on its right,
 - the pivot sits between them, in its **final sorted position**.

3. Now the left range and the right range are independent smaller problems. Repeat on each.

5.2 Recursive version vs the version used here

Normally, step 3 is done by the function calling itself on the left and right ranges. Without functions, we keep our own **to-do list** of ranges that still need partitioning. This to-do list is a **stack** (last in, first out), and we build it with a plain array:

- `stack[]` stores pairs: `low, high`.
- Push a pair = "remember this range for later".
- Pop a pair = "take the most recently remembered range and work on it".

Loop until the stack is empty, meaning no range is left to work on.

5.3 The partition method used (Lomuto scheme)

Range is `[low .. high]`, pivot is `arr[high]`.

- Keep a boundary variable `i` that marks the end of the "smaller than pivot" area. It starts at `low - 1` (the area is empty).
- Scan `j` from `low` to `high - 1`. Whenever `arr[j] < pivot`, grow the small area (`i++`) and swap `arr[i]` with `arr[j]`, which moves the small value into the small area.
- At the end, swap the pivot (at `high`) with `arr[i + 1]`. The pivot lands right after the small area, which is its final position.

Picture during a scan:

```
[ smaller than pivot ][ bigger or equal ][ not yet checked ][ pivot ]
low ..... i         i+1 ..... j-1      j ..... high-1      high
```

5.4 Source code

```
#include <iostream>
using namespace std;

int main() {
    const int n = 8;
    int arr[n] = {7, 2, 1, 6, 8, 5, 3, 4};

    int stack[100];    // our own stack of ranges (holds low, high, low, high, ...)
    int top = -1;      // index of the top item; -1 means the stack is empty

    cout << "Before sorting: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    // Push the whole array as the first range to work on
    top++; stack[top] = 0;        // low
    top++; stack[top] = n - 1;    // high

    // Keep working while there are ranges left in the stack
    while (top >= 0) {

        // Pop a range (pop high first, because it was pushed last)
        int high = stack[top];
        top--;
        int low = stack[top];
        top--;
```

```

// PARTITION the range [low .. high] around the pivot
int pivot = arr[high];      // pivot = last element of the range
int i = low - 1;            // end of the "smaller than pivot" area

for (int j = low; j < high; j++) {
    if (arr[j] < pivot) {
        i++;                // grow the small area
        int temp = arr[i];  // swap arr[i] and arr[j]
        arr[i] = arr[j];
        arr[j] = temp;
    }
}

// Put the pivot into its final place, right after the small area
int temp = arr[i + 1];
arr[i + 1] = arr[high];
arr[high] = temp;

int p = i + 1;              // p = final position of the pivot

// If the left side [low .. p-1] has 2 or more elements, remember it
if (p - 1 > low) {
    top++; stack[top] = low;
    top++; stack[top] = p - 1;
}

// If the right side [p+1 .. high] has 2 or more elements, remember it
if (p + 1 < high) {
    top++; stack[top] = p + 1;
    top++; stack[top] = high;
}

cout << "After sorting: ";
for (int i = 0; i < n; i++) {
    cout << arr[i] << " ";
}
cout << endl;

return 0;
}

```

5.5 How the code works, part by part

The **stack** (`stack[100]` and `top`) `top` is the position of the last item pushed. `top = -1` means empty.

- Push: `top++; stack[top] = value;`
- Pop: `value = stack[top]; top--;`

Because a range needs two numbers, we always push `low` first then `high`, and always pop in the reverse order: `high` first, then `low`. (Size 100 is plenty for small examples; the stack only ever holds a few ranges.)

The main `while (top >= 0)` "While there is still a range to sort, take one and process it."

Partition part Follows the scheme explained in section 5.3. After this part, the pivot is in its **final correct position** `p`, with smaller values to its left and bigger (or equal) values to its right. Neither side is sorted internally yet, just separated correctly.

Pushing the sub-ranges

- Left range is `low .. p-1`. It has 2 or more elements only if `p - 1 > low`. A range of 0 or 1 element is already sorted, so we skip it.

- Right range is $p+1 \dots \text{high}$. Same reasoning with $p + 1 < \text{high}$.

Each partition places one pivot permanently and creates up to two smaller ranges. Eventually all ranges become size 0 or 1, the stack empties, and the array is sorted.

5.6 Dry run (trace) with {7, 2, 1, 6, 8, 5, 3, 4}

Start: stack = (0, 7) .

Range (0, 7), pivot = 4

j	arr[j]	arr[j] < 4?	Action	Array
0	7	no	nothing	7 2 1 6 8 5 3 4
1	2	yes	i=0, swap arr[0], arr[1]	2 7 1 6 8 5 3 4
2	1	yes	i=1, swap arr[1], arr[2]	2 1 7 6 8 5 3 4
3	6	no	nothing	2 1 7 6 8 5 3 4
4	8	no	nothing	2 1 7 6 8 5 3 4
5	5	no	nothing	2 1 7 6 8 5 3 4
6	3	yes	i=2, swap arr[2], arr[6]	2 1 3 6 8 5 7 4

Place pivot: swap $\text{arr}[i+1] = \text{arr}[3]$ with $\text{arr}[7]$ giving 2 1 3 4 8 5 7 6. Pivot 4 is at index $p = 3$, which is its final place.

Left is (0, 2) and right is (4, 7) . Both are pushed. Stack (bottom to top): (0, 2) (4, 7) .

Pop (4, 7), pivot = 6 (array section: 8 5 7 6)

- $8 < 6$? no. $5 < 6$? yes, $i = 4$, swap $\text{arr}[4]$ and $\text{arr}[5]$: section becomes 5 8 7 6 .
- $7 < 6$? no.
- Place pivot: swap $\text{arr}[5]$ with $\text{arr}[7]$: 2 1 3 4 5 6 7 8 . Pivot 6 is at $p = 5$.
- Left (4, 4) has 1 element, skip. Right (6, 7) has 2 elements, push.

Pop (6, 7), pivot = 8 (section: 7 8)

- $7 < 8$? yes, $i = 6$, swap $\text{arr}[6]$ with itself.
- Place pivot: swap $\text{arr}[7]$ with itself. $p = 7$. Both sides are empty. Nothing pushed.

Pop (0, 2), pivot = 3 (section: 2 1 3)

- $2 < 3$ yes, $1 < 3$ yes (each swaps with itself). Pivot 3 is already at index 2, $p = 2$.
- Left (0, 1) has 2 elements, push. Right is empty.

Pop (0, 1), pivot = 1 (section: 2 1)

- $2 < 1$? no. i stays -1 .
- Place pivot: swap $\text{arr}[0]$ with $\text{arr}[1]$: 1 2 . $p = 0$. Nothing to push.

Stack is empty. Final array: 1 2 3 4 5 6 7 8

5.7 Summary

Property	Value
Best / Average	$O(n \log n)$
Worst case	$O(n^2)$ (happens when pivots are always the smallest or biggest, e.g. an already sorted array with last-element pivot)
Extra memory	$O(\log n)$ typically for the stack (this version uses a fixed array)
Stable	No
Good for	General purpose sorting, usually the fastest in practice

Beginner trap

With the last element as pivot, an already sorted array is the **worst case**. Real-world implementations pick a middle or random pivot to avoid this. It is left out here to keep the code simple.

6. Side by Side Comparison

Algorithm	Best	Average	Worst	Extra memory	Stable	Main idea
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Swap neighbours, biggest bubbles to the end
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Pick the smallest, put it at the front
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Insert each item into the sorted left part
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	Split, then merge sorted pieces
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Partition around a pivot

Which one should I use?

- Learning the basics: Bubble, Selection, Insertion.
- Small or nearly sorted data: Insertion Sort.
- Big data with guaranteed speed, or stability needed: Merge Sort.
- Big data, general speed: Quick Sort.
- In real projects you would normally call the built-in `std::sort` from `<algorithm>`. Writing these by hand is for understanding.

7. Common Mistakes Checklist

- Off-by-one errors in loop limits ($n - 1$ vs n). Always ask "what is the last valid index?" (it is $n - 1$).
- Writing `while (arr[j] > key && j >= 0)` in Insertion Sort. The `j >= 0` check must come **first**, otherwise `arr[-1]` is read.

- Forgetting the `temp` variable when swapping, which causes duplicated values and lost values.
- In Selection Sort, storing the smallest **value** instead of its **index**, so you cannot swap it.
- In Merge Sort, forgetting to copy `temp` back into `arr`, or forgetting to clamp `mid` and `right` to `n`.
- In Quick Sort, popping `low` and `high` in the wrong order from the stack.

8. Practice Ideas

1. Change the arrays to your own numbers (include duplicates and negatives) and predict the output before running.
2. Add a `cout` inside the outer loop of each program to print the array after every pass, and compare with the traces above.
3. Change the sort to **descending** order (hint: flip one comparison sign in each program).
4. Count how many comparisons each algorithm does on the same array using a counter variable.
5. Try an already sorted array and a reverse sorted array, and see which algorithms slow down.